File Reference: BRANCHING.md

A file-specific explanation covering purpose, owner layer, metadata, source details, implementation signals, source excerpts, and safe-change rules.

Item	Explanation
File	BRANCHING.md
Category	Documentation or text control file
Folder role	Repository root. Usually an entry file, app config, public website file, package manifest, or top-level documentation.
Size	3,052 bytes
Extension	.md
MIME type	text/markdown
Text lines	55

Why this file exists

- Human-readable branch model explaining development, main, feature branches, and release flow.

How it is used

Repository root. Usually an entry file, app config, public website file, package manifest, or top-level documentation.

How this file participates in the system

This generated section reads the actual file and points out line numbers, declarations, endpoints, events, source excerpts, and debugging cues.

Important implementation signals

Item	Explanation
Git command at line 19	Repository state, publishing, branch checks, or release automation. Evidence: git checkout development
Git command at line 20	Repository state, publishing, branch checks, or release automation. Evidence: git pull origin development
Git command at line 21	Repository state, publishing, branch checks, or release automation. Evidence: git checkout -b feature/my-change
Git command at line 29	Repository state, publishing, branch checks, or release automation. Evidence: git add .
Git command at line 30	Repository state, publishing, branch checks, or release automation. Evidence: git commit -m "Describe the change"
Git command at line 31	Repository state, publishing, branch checks, or release automation. Evidence: git push -u origin feature/my-change

Representative opening snippet

```
     1: # Branching Workflow
     2:
     3: This repository uses a protected release-style workflow.
     4:
     5: ## Branches
     6:
     7: - `main` is the consumer branch. It should contain only finished, tested, release-ready code.
     8: - `development` is the integration branch. It collects completed work from feature branches before
release.
     9: - `feature/<short-name>` branches are used for individual app changes, experiments, fixes, or UI
improvements.
    10:
    11: `main` must only receive changes from `development`. Feature branches, Codex branches, hotfix branches,
and experiment branches must merge into `development` first. After `development` has collected and verified the
changes, `development` is the only branch that may be merged into `main`.
    12:
    13: ## Normal Change Flow
    14:
    15: 1. Start from the latest `development`.
    16: 2. Create a focused feature branch:
    17:
```

```
   18: ```powershell
   19: git checkout development
   20: git pull origin development
   21: git checkout -b feature/my-change
   22: ```
   23:
   24: 3. Update the branch register in `README.md` with the new branch name, purpose, source branch, merge
target, and status.
   25: 4. Make and test the change.
   26: 5. Commit and push the feature branch:
   27:
   28: ```powershell
   29: git add .
   30: git commit -m "Describe the change"
   31: git push -u origin feature/my-change
   32: ```
   33:
   34: 6. Merge or pull-request the feature branch into `development`.
   35: 7. Update the branch register status when the branch is merged, closed, or released.
   36: 8. Test `development`.
   37: 9. Before merging `development` into `main`, bump `package.json` to the next release version.
   38: 10. Merge `development` into `main` only when the app is ready for consumers. No other source branch
should target `main`.
   39: 11. Push `main` to build and publish the `builder-v<version>` release automatically, or create a release
tag from `main`, for example `builder-v<next-version>`.
   40:
   41: ## Branch Register Rule
   42:
   43: The README is the human-facing branch register. Every new branch should have a row in `README.md` before
the branch is pushed. This keeps local users, GitHub visitors, and future Codex sessions aligned on what each
branch is for.
   44:
   45: ## Release Rule
   46:
   47: Do not make app changes directly on `main`. `main` should move only after `development` has collected and
 verified the changes.
   48:
   49: Pull requests into `main` must use `development` as the source branch. The repository includes
`.github/workflows/main-branch-gate.yml`, which fails pull requests into `main` from any other branch. GitHub
branch protection should also be enabled for `main` so direct pushes are blocked and release changes go through
the development-to-main pull request path.
   50:
   51: Installed apps detect updates by comparing their current app version with the latest GitHub Release
version. A `main` push must therefore carry a new `package.json` version. The release workflow rejects a `main`
push if the matching `builder-v<version>` tag already exists.
   52:
   53: ## Current Practice
   54:
   55: For Codex-assisted work, new app changes should be made on a focused `codex/...` or `feature/...` branch,
 merged into `development`, and only then released through `development` into `main`.
```

Safe change checklist

- Confirm which app, website, installer, workflow, or document consumes this file.

- Change it on a feature branch from development.

- Regenerate docs if the file purpose, API surface, or behavior changes.

- Run the smallest relevant smoke test before merging into development.